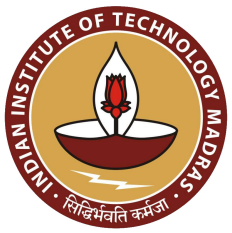

Indian Institute of Technology Madras

Department of Physics

Adaptive Replica Exchange MCMC for Job Shop Scheduling

Rohith R

Supervisor: Dr. Ramprasath S
Department of Electrical Engineering

A report submitted in partial fulfilment of the requirements of the Indian Institute of Technology Madras for the degree of Bachelors of Technology in *Engineering Physics*

June 16, 2026

Declaration

I, Rohith R, of the Department of Physics, Indian Institute of Technology, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that if failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

I give consent for my work to be made available more widely to members of IITM and public with interest in teaching, learning and research.

Rohith R

June 16, 2026

Abstract

We propose an approach to solving the $n \times m$ Job Shop Scheduling Problem (JSSP) using an adaptive version of Replica Exchange Markov Chain Monte Carlo method, which, to the best of our knowledge, is being applied to this problem for the first time. A novel neighborhood generation is proposed that enables the generation of valid neighbors with high probability and an efficient incremental update to the longest-path calculation, avoiding the need to recompute paths from scratch after small perturbations. By incorporating adaptive temperature control into the replica exchange framework, we facilitate an efficient random walk of states from colder to hotter chains, leading to improved sampling efficiency in the coldest chain. The algorithm is implemented in a parallel programming framework using a multithreaded system to fully leverage computational resources. Experimental results show that our method achieves competitive performance on standard JSSP instances. The codes are available at

Keywords: Job Shop Scheduling, Parallel Tempering, Combinatorial Optimization, Incremental Topological Sorting, Order Theory, Simultaneous Triangularization.

Acknowledgments

I would like to sincerely thank Dr. Ramprasath S for his invaluable guidance and support, especially for providing insightful ideas whenever I encountered obstacles, and for his unwavering understanding throughout the project. His expertise and encouragement have been a source of great motivation for me. I truly appreciate his patience and dedication to my growth.

I would also like to thank my friends for their attentive listening, valuable feedback, and constant encouragement throughout the course of this project. Their moral support and thought-provoking suggestions were integral to refining my ideas and pushing the boundaries of my work.

I would also like to express my heartfelt gratitude to the department for providing me with the opportunity to undertake this project

Contents

List of Figures

Chapter 1

Introduction and Literature Review

1.1 Introduction

The **Job Shop Scheduling Problem (JSSP)** (1) is a widely studied and thriving area of research in the fields of computer science, operations research, and optimization. It involves assigning a set of jobs, each consisting of a sequence of operations (tasks), to a set of machines with the goal of optimizing objectives such as minimizing the total completion time (makespan), minimizing lateness, or maximizing machine utilization. JSSP is known for its combinatorial complexity and is classified as an NP-hard problem, making it computationally challenging as the problem size grows. Due to its practical relevance in manufacturing, logistics, and production planning, a vast array of exact algorithms, heuristic methods, and metaheuristic techniques like genetic algorithms, simulated annealing, and reinforcement learning have been developed to tackle it. The problem continues to inspire new research directions, particularly in areas such as hybrid approaches, machine learning-assisted scheduling, and quantum computing solutions (2) (3) (4) (5).

In this work, we propose a novel approach to solving the $n \times m$ JSSP using an adaptive version of Parallel Tempering (also known as Replica Exchange Markov Chain Monte Carlo). This technique allows for the optimized exchange of information between replicas operating at different temperatures, where higher-temperature chains explore the solution space more freely and lower-temperature chains focus on exploitation. By adaptively tuning the temperature ladder and swap acceptance probabilities, our method maintains diversity while ensuring convergence toward high-quality solutions.

A central component of our framework is a custom-designed neighbor generation technique, which draws conceptual connections to simultaneous triangularization and principles from order theory. This methodology enables efficient, incremental updates to the topological sorting order that represents the current scheduling configuration. By leveraging this structure, we compute the longest path—corresponding to the makespan—through the evolving directed acyclic graph in an efficient and consistent manner. Together, these innovations yield a robust and scalable approach for tackling the complexities of JSSP.

1.2 Literature Review

To the best of our knowledge, this is the first application of parallel tempering to solve the Job Shop Scheduling Problem (JSSP). While parallel tempering has not been previously explored in this context, several other probabilistic optimization methods have been employed. In (6), simulated annealing is used to train a selection hyper-heuristic. A hybrid metaheuristic called AntGenSA, combining Ant Colony Optimization, Simulated Annealing, and Genetic Algorithms, is proposed in (7). Simulated annealing is also used in (8) to find non-dominated solutions to the bi-objective flexible Job Shop Problem, an extension of JSSP. In (9), ant colony optimization is dynamically adapted using genetic algorithms to suit the problem structure.

Parallel tempering is a generic Markov Chain Monte Carlo (MCMC) method designed to efficiently sample from multi-modal distributions by enabling better mixing across modes. Its performance heavily depends on the temperature ladder, with optimal efficiency achieved when the swap acceptance probability (SAP) is uniform across temperature pairs. It has been shown that the optimal SAP for systems with (piecewise) constant specific heat capacity is approximately 23% (10). An adaptive version of parallel tempering, which tunes both the inverse temperatures and the proposal kernel of the random-walk Metropolis sampler to improve local exploration, is proposed in (11); this approach also parameterizes the inverse temperatures—a technique we also adopt. In (12), the temperature differences between chains are defined in terms of the log-temperature spacing, ensuring correct ordering during adaptation, although their method aims for uniform SAP rather than a target value. The approach in (13) extends adaptivity further by dynamically adding or removing replicas to balance resource usage and execution time, with implementation on an FPGA platform.

Maintaining topological order and longest paths are two central problems in dynamic graph algorithms (14). Recent advances include the use of algorithms-with-predictions to design data structures for incremental topological ordering and cycle detection (15). However, maintaining the longest paths introduces additional challenges, as updates to the topological order do not necessarily imply changes to the longest paths. The work in (16) and a series of publications by Madraki et al. (17; 18; 19) propose efficient algorithms for updating longest paths in perturbed DAGs. These incremental algorithms typically achieve a time complexity of $\mathcal{O}(||\Delta|| + |\Delta| \log |\Delta|)$, where $|\Delta|$ is the number of affected nodes and $||\Delta||$ is the total number of incident edges on those nodes.

Chapter 2

Background

2.1 Job Shop Scheduling Problem

In this work, we focus on the $n \times m$ class of Job Shop Scheduling Problem (JSSP) instances, where n denotes the number of jobs and m denotes the number of machines. Each job is composed of exactly m operations, one for each machine. Formally, there are n jobs $\mathcal{J} = \{J_1, \ldots, J_n\}$ and m machines $M = \{m_1, \ldots, m_m\}$. Each job $J_i \in \mathcal{J}$ consists of a sequence of operations, denoted by $J_i = \{o_{i1}, \ldots, o_{im}\}$, where each operation must be processed in a specific order. That is, operation o_{in} must be completed before $o_{i,n+1}$.

Each operation o_{in} is assigned to a machine according to a mapping $\sigma(o_{in}) \in M$ and must be processed non-preemptively for p_{in} units of time. A feasible schedule specifies a total order over operations assigned to each machine, respecting the within-job operation sequence constraints. Let τ_{in} denote the starting time of operation o_{in}. The makespan of a schedule—the total time required to complete all jobs—is given by:

$$\text{Makespan} = \max_{i,n} \left(\tau_{in} + p_{in}\right).$$

The primary objective in most formulations of the JSSP is to find a schedule that minimizes this makespan.

2.1.1 Disjunctive Graph Formulation

The *Disjunctive Graph* $\mathcal{G} = (V, E_C \cup E_D)$ provides a convenient mathematical model to represent the JSSP as a precedence-constrained scheduling problem.

- V is the set of nodes, where each node corresponds to an operation o_{in}, augmented with a unique source node s and a sink node t.
- E_C is the set of **conjunctive arcs**, representing the fixed precedence constraints between successive operations within the same job. Specifically, there is a directed arc from o_{in} to $o_{i,n+1}$ for all i and n.
- E_D is the set of **disjunctive arcs**, connecting pairs of operations assigned to the same machine. These arcs are initially undirected: deciding a direction for

each disjunctive arc corresponds to selecting the processing order of operations sharing the same machine.

The weight of each arc corresponds to the processing time of the operation represented by its source node. A schedule is fully specified by orienting all disjunctive arcs to produce a directed acyclic graph (DAG). The schedule is **feasible** if the resulting directed graph is acyclic. The makespan then corresponds to the length of the longest path from the source s to the sink t in the DAG.

For example, consider a 3×3 JSSP instance:

Job 1	(1,3)	(2,2)	(3,2)
Job 2	(1,2)	(2,1)	(3,4)
Job 3	(2,4)	(1,3)	(3,2)

Here, the first entry in each tuple represents the machine required, and the second entry represents the processing time. Each operation is assigned a unique identifier based on its position in the rectangular array. For example, the second operation of Job 2 is assigned ID 4.

Figure 2.1 illustrates the disjunctive graph representation of the example instance (edge weights are omitted for clarity).

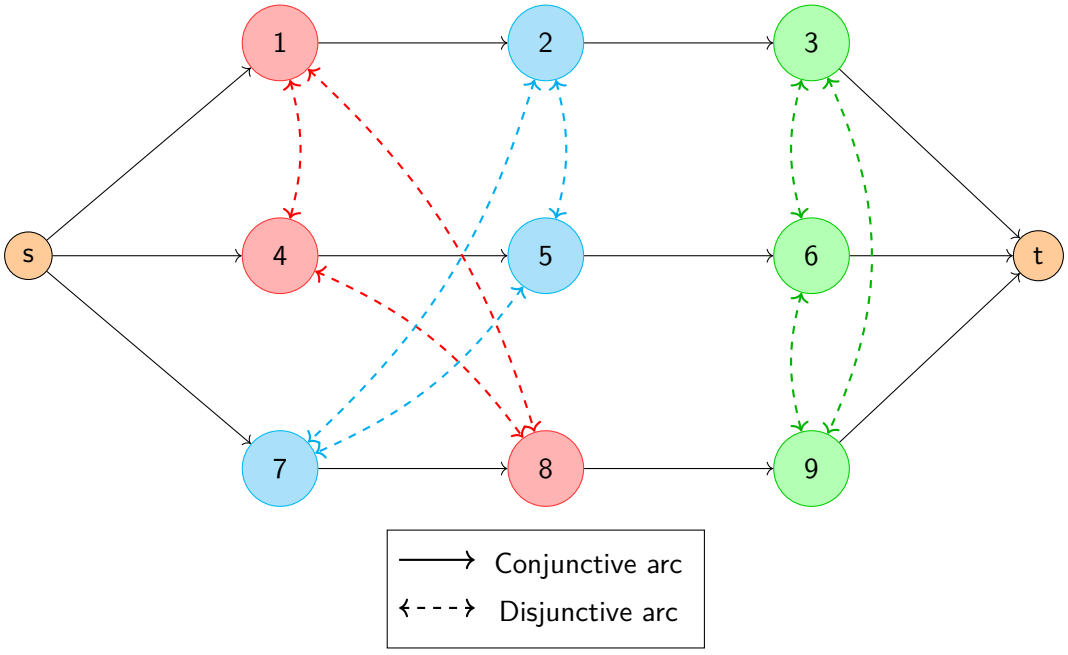

Figure 2.1: Disjunctive graph representation for a 3×3 Job Shop Scheduling Problem.

2.2 Replica Exchange MCMC

Markov Chain Monte Carlo (MCMC) algorithms, such as the Metropolis–Hastings algorithm, have been widely used to sample from a probability distribution when its

density is known up to a normalization constant. Simulated Annealing is a modified version of the Metropolis–Hastings algorithm used exclusively in the context of optimization problem that samples from the Boltzmann distribution using a temperature schedule that decreases with iterations. However, it often struggles in multimodal energy landscapes, where the algorithm can get trapped in local minima.

Population-based MCMC algorithms, such as Replica Exchange (also known as Parallel Tempering), offer a more robust alternative due to their inherent ability to escape local minima and explore multiple modes of the probability distribution. This approach runs several Markov chains in parallel, each sampling from a tempered version of the posterior distribution π. The tempered distribution for temperature T is given by:

$$\pi_T(\tilde{\theta}) \propto L(\tilde{\theta})^{1/T} p(\tilde{\theta}),$$

where $L(\tilde{\theta})$ is the likelihood and $p(\tilde{\theta})$ is the prior distribution. At higher temperatures, the likelihood surface becomes flatter and broader, making the distribution easier to explore using MCMC. This is illustrated in Figure 2.2.

A set of N chains is assigned temperatures along a ladder:

$$T_1 < T_2 < \cdots < T_N.$$

Each chain samples from its respective tempered distribution π_{T_i} using an MCMC algorithm. At pre-determined intervals, swaps are proposed between adjacent chains i and $j = i + 1$, and accepted with probability

$$\text{SAP} = \min\left\{ \left(\frac{L(\tilde{\theta}_i)}{L(\tilde{\theta}_j)} \right)^{\beta_j - \beta_i}, 1 \right\} \tag{2.1}$$

where $\tilde{\theta}_i$ is the current position of the i-th chain and $\beta_i = 1/T_i$ is its inverse temperature. If a swap is accepted, the chains exchange their positions in parameter space: chain i moves to $\tilde{\theta}_j$, and chain j to $\tilde{\theta}_i$.

Because high-temperature chains can traverse the entire state space, including all modes, successful swaps propagate this exploratory capability to lower-temperature chains. This mechanism allows the coldest chain to efficiently explore the target distribution π.

In the context of optimization, the likelihood $L(\tilde{\theta})$ is replaced by a Boltzmann distribution, where the energy function corresponds to the objective function being minimized.

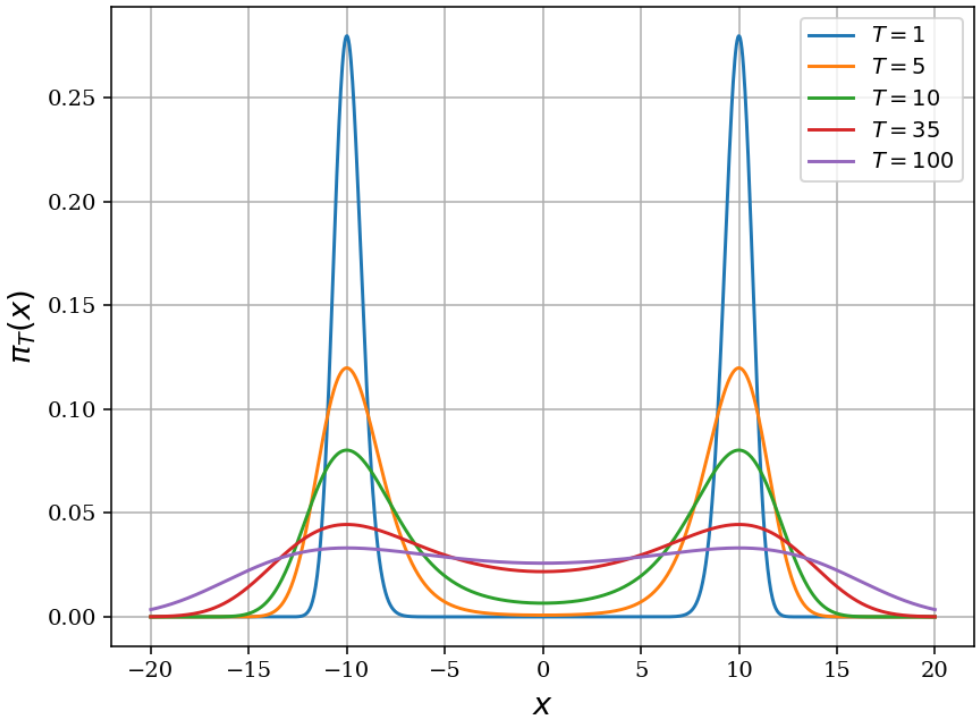

Figure 2.2: Tempered versions of a one-dimensional distribution. At high temperature values, the peaks broaden, allowing an MCMC chain to quickly explore all modes starting from any position.

2.3 Bounded Incremental Complexity

A central challenge in optimization via search methods is the cost of evaluating the black-box objective function at each point. In our setting, this cost corresponds to computing the longest s-t path in a directed acyclic graph (DAG) induced by a given orientation of disjunctive arcs.

The Standard Longest Path Algorithm (SLPA) addresses this by performing a topological sort of the nodes and applying the principle of optimality to compute the longest s-t path in $\mathcal{O}(|E| + |V|)$ time. We will review the details of SLPA in subsequent sections.

However, in our algorithm, each move in the search space involves only a small perturbation to the current orientation $\mathbf{x}$, resulting in a neighboring state $\mathbf{x} + \epsilon$. Instead of recomputing the objective from scratch at $\mathbf{x}+\epsilon$, we can leverage information from $\mathbf{x}$ to compute the new value more efficiently.

Traditional worst-case analysis expresses computational cost as a function of input size and often fails to capture the efficiency gains of incremental algorithms—those that reuse solutions from nearby instances. In fact, many known incremental algorithms run in time asympototically no better, in the worst-case, than the time required to perform the computation from scratch.

To better characterize such scenarios, we adopt the framework of *Bounded Incremental Computation (BIC)* introduced by G. Ramalingam (20). BIC evaluates the cost of an incremental algorithm in terms of the size of changes to the input and the resulting output. Specifically, an incremental algorithm is *bounded* if, for all inputs $\mathbf{x}$ and allowed perturbations ϵ, its execution time depends only on ϵ and Δ—the change in the output induced by ϵ.

In this framework, SLPA is regarded as an unbounded algorithm since its runtime scales with the entire input size rather than the extent of the change. However, we will later introduce a bounded algorithm that efficiently recalculates the longest path under our defined notion of perturbation.

2.4 Review of Graph Theory

2.4.1 Standard Longest Path Algorithm

Definition 2.4.1. *A **topological sorting** of nodes in a DAG $\mathcal{G} = (V, E)$ is defined as an ordering $L = \{n_1, n_2, \ldots, n_N\}$ such that for any $i < j$, it holds that $n_i \prec n_j$, where $N = |V|$ and $\prec$ is the reachability relation—i.e., $n_i \prec n_j$ implies there exists a directed path from n_i to n_j.*

Given a DAG and an initial node s, the longest path from s to every other node can be computed efficiently using the topological order. The main idea is to iterate through the nodes in topological order and use dynamic programming to propagate the longest distances from s. This approach relies on the principle of optimality: the longest path to a node must include the longest path to one of its predecessors.

Algorithm 1 Longest Path in Directed Acyclic Graph (DAG)

Input: DAG $\mathcal{G} = (V, E)$, edge weights $w : E \to \mathbb{R}$, source node s
Output: Longest distance $\text{dist}[v]$ from s to all $v \in V$

```
Initialize dist[v] ← −∞ for all v ∈ V; dist[s] ← 0
Compute a topological ordering L = {n_1, n_2, ..., n_|V|}
for each node u in order L do
    for each edge (u, v) ∈ E do
        if dist[v] < dist[u] + w(u, v) then
            dist[v] ← dist[u] + w(u, v)
        end if
    end for
end for
return dist
```

This algorithm runs in $\mathcal{O}(|V| + |E|)$ time, since all nodes and edges must be visited. In our case since all the weights are strictly positive the length of the longest path to the nodes also present a topological sort of the DAG.

2.4.2 Matrix Theory of Directed Acyclic Graphs

Definition 2.4.2. *The **adjacency matrix** $A_{\mathcal{G}}$ of a finite directed graph $\mathcal{G} = (V, E)$ with $|V| = N$ vertices is an $N \times N$ square matrix. When the context is clear, we omit the subscript $\mathcal{G}$. Each entry A_{ij} indicates whether an edge exists from vertex i to vertex j:*

$$A_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Definition 2.4.3. ***Schur Decomposition**: For any square matrix $A \in \mathbb{C}^{N \times N}$, there exists a unitary matrix Q and an upper triangular matrix U such that*

$$A = QUQ^{\dagger},$$

where $Q^{\dagger}$ denotes the conjugate transpose of Q. Since A and U are similar matrices and U is upper triangular, the diagonal entries of U are the eigenvalues of A.

Few of the useful remarks and concepts are mentioned below.

1. **Walks and Matrix Powers:** For any adjacency matrix A, the (i, j) entry of A^k counts the number of distinct walks of length exactly k from node i to node j:
$$(A^k)_{ij} = \text{number of walks of length } k \text{ from } i \text{ to } j.$$

2. **Nilpotency of DAG Adjacency Matrices:** The adjacency matrix A of a DAG is *nilpotent*. Since the longest possible path in a DAG contains at most $N-1$ edges (where each node appears at most once), by property 1, $A^k = 0$ for all $k \geq N$. Conversely, if $A \in \{0, 1\}^{N \times N}$ is nilpotent, then the corresponding graph must be acyclic.

3. **Eigenvalue Characterization:** From property 2, it follows that all eigenvalues of the adjacency matrix A of a DAG are zero. In contrast, the adjacency matrix of a directed graph with cycles must have at least two non-zero eigenvalues as for graphs without self-loops, $\text{tr}(A) = 0$, which means that non-zero eigenvalues must sum to zero.

4. **Topological Sorting and Triangularization:** For any DAG, there exists a topological ordering of the nodes $L = \{n_1, n_2, \ldots, n_N\}$. Using this ordering, we can transform the adjacency matrix A into an upper triangular matrix U via a permutation matrix P:
$$P^T AP = U \implies A = PUP^T,$$
where U is upper triangular. This is the Schur decomposition of A. The graphs represented by A and U are *isomorphic*, differing only in node labeling. We say that *P triangularizes A if and only if the permutation mapping π encoded by P forms a valid topological sorting order for $\mathcal{G}_A$.*

Definition 2.4.4. *We say an invertible matrix P **simultaneously triangularizes** two matrices A and B if both $P^T AP$ and $P^T BP$ are upper triangular matrices.*

The concept of *simultaneous triangularization* applies when A and B represent two graphs on the same node set with no shared edges. A matrix P that simultaneously triangularizes A and B encodes the topological sorting order of the graph formed by the union of the edges from $\mathcal{G}_\mathcal{A}$ and $\mathcal{G}_\mathcal{B}$.

A key result is that if A and B satisfy $A[A, B] = 0$ and $[A, B]B = 0$, they can be simultaneously triangularized. This condition is satisfied when A and B commute (21).

2.5 Order Theory

Order Theory is a branch of mathematics that provides a formal framework for understanding the concept of order through binary relations. The following definitions will be useful in the subsequent sections.

Definition 2.5.1. *A **partially ordered set** (or **poset**) is a pair $L = (X, \leq)$, where X is a set and $\leq$ is a* partial order *on X. A partial order is a binary relation that is* reflexive *(every element is comparable to itself),* antisymmetric *(if $x \leq y$ and $y \leq x$, then $x = y$), and* transitive *(if $x \leq y$ and $y \leq z$, then $x \leq z$). The term "partial" indicates that not all pairs of elements in X need to be comparable under $\leq$.*

When the partial order is clear from context, the set X is often referred to as the poset itself.

Definition 2.5.2. *A **total order** (or **linear order**) is a special type of partial order in which every pair of elements is comparable. That is, for all $x, y \in X$, either $x \leq y$ or $y \leq x$. Thus, a total order is a partial order that is also* total.

Definition 2.5.3. *A **linear extension** of a poset $L = (X, \leq)$ is a total order $\leq'$ on X such that for all $x, y \in X$, if $x \leq y$ in the original poset, then $x \leq' y$ in the extension. Linear extensions provide a way to represent partial orders as sequences where the original ordering constraints are preserved.*

The directed edges in a Directed Acyclic Graph (DAG) naturally induce a poset, where the vertices represent elements and the reachability relation defines the partial order. A topological ordering of the DAG corresponds to a linear extension of this poset.

Definition 2.5.4. *A **chain** in a poset is a subset of elements in which every pair of elements is comparable under the partial order. In contrast, an **antichain** is a subset in which no two elements are comparable. Chains represent fully ordered subsets, while antichains represent sets of mutually incomparable elements.*

Definition 2.5.5. *The **width** of a poset is the size of its largest antichain. This concept is central in the study of posets, especially in the context of Dilworth's theorem, which states that in any finite poset, the size of the largest antichain equals the minimum number of chains needed to cover the set.*

The concepts of *chain* and *antichain* become particularly useful when interpreting conjunctive and disjunctive arcs through the lens of order theory.

Chapter 3

Adaptive Parallel Tempering

3.1 Overview

Now we will formally describe Parallel Tempering and its adaptive version. PT operates over the product space X^N, where X is the state space of the problem and N is the number of chains (or replicas). Let the state at the k-th iteration be

$$\mathbf{x}_k = (X_k^{(1)}, X_k^{(2)}, \ldots, X_k^{(N)}),$$

where each $X_k^{(i)}$ samples from a tempered version of the target distribution π, parameterized by inverse temperatures $\boldsymbol{\beta} = (\beta^{(1)}, \ldots, \beta^{(N)})$ such that $\beta^{(1)} > \cdots > \beta^{(N)} > 0$.

The joint stationary distribution is:

$$\boldsymbol{\pi}_{\boldsymbol{\beta}}(x^{(1)}, \ldots, x^{(N)}) = \prod_{i=1}^{N} \frac{\pi^{\beta^{(i)}}(x^{(i)})}{Z(\beta^{(i)})},$$

where $Z(\beta^{(i)})$ is the normalization constant at the inverse temperature $\beta^{(i)}$. The algorithm proceeds in two steps: (1) each replica samples locally for N_{sweep} steps, and (2) a pair of neighboring replicas is selected uniformly at random, and a swap is proposed with swap acceptance probability (SAP) given by Equation 2.1

Adaptive Parallel Tempering improves PT by dynamically tuning the inverse temperature ladder to enhance inter-chain communication. Optimal exploration is facilitated by adapting the temperature spacings to achieve a target SAP (typically between 0.2 and 0.4), which encourages uniform mixing across chains.

To enforce the constraint $\beta^{(1)} > \cdots > \beta^{(N)}$, we parameterize the inverse temperatures using parameters $\rho_i^{(l)}$:

$$\beta_i^{(1)} \triangleq \frac{1}{T_{\text{low}}}, \quad \beta_{i+1}^{(l)} = \beta_i^{(l)} \cdot \sigma(\rho_i^{(l)}),$$

where $\sigma(z) = \dfrac{1}{1+\exp(-z)}$ is the sigmoid function. Only $\beta^{(1)}$ is fixed; all others are adapted via the $\rho_i^{(l)}$ values. The update rule is:

$$\rho_{i+1}^{(l)} = \Pi_d \left(\rho_i^{(l)} + \gamma(i) \cdot (\text{target\_sap} - \text{sap}_l) \right),$$

where sap_l is the swap acceptance probability between chain l and $l+1$ and Π_d projects onto $[-d, \infty)$ with $d > 0$ to ensure numerical stability. The function $\gamma(i)$ is a diminishing adaptation rate:

$$\gamma(i) = c \cdot (1+i)^{-\eta},$$

following (22), who showed that diminishing adaptation ensures ergodicity despite violating detailed balance during adaptation which is crucial to ensure that a MCMC sampler will converge to a target distribution.

The intuition behind this update is that if the observed SAP is higher than the target, the chains are too close in temperature space and need to be separated, which is achieved by reducing ρ_i^l. Conversely, if SAP is too low, increasing ρ_i^l brings the temperatures closer together.

Algorithm 2 Adaptive Parallel Tempering Block

Input: $\mathbf{X}_i = (X_i^{(1)}, \ldots, X_i^{(N)})$, $\boldsymbol{\beta}^{(l)}$, $\boldsymbol{\rho}^{(l)}$, $\gamma(i)$, target_sap, $\{\text{sap}_i\}_i$, N_{sweep}
Output: Updated states $\mathbf{X}_{i+1}$, updated $\boldsymbol{\beta}^{(l)}$, updated $\boldsymbol{\rho}^{(l)}$

for $l = 1$ to N **do** (IN PARALLEL)
  for $s = 1$ to N_{sweep} **do**
    Perform local MCMC update of $X_i^{(l)}$ at inverse temperature $\beta^{(l)}$
  end for
end for
Randomly choose $j \in \{1, \ldots, N-1\}$
Propose swap between $X_i^{(j)}$ and $X_i^{(j+1)}$
Compute SAP: $\text{sap}_j \leftarrow \min\left(1, \dfrac{\pi^{\beta^{(j)}}(X_i^{(j+1)})\pi^{\beta^{(j+1)}}(X_i^{(j)})}{\pi^{\beta^{(j)}}(X_i^{(j)})\pi^{\beta^{(j+1)}}(X_i^{(j+1)})}\right)$
Accept swap with probability sap_j and update $\mathbf{X}_{i+1}$
Update $\rho_{i+1}^{(j)} \leftarrow \Pi_d \left(\rho_i^{(j)} + \gamma(i) \cdot (\text{target\_sap} - \text{sap}_j) \right)$
Set $\beta_{i+1}^{(1)} \leftarrow \frac{1}{T_{\text{low}}}$
for $l = 2$ to $N-1$ **do**
  $\beta_{i+1}^{(l+1)} \leftarrow \beta_{i+1}^{(l)} \cdot \dfrac{1}{1+\exp(-\rho_{i+1}^{(l)})}$
end for

3.2 Examples

3.2.1 Sampling from a Mixture of Two Gaussians

To demonstrate the effectiveness of adaptive parallel tempering, we consider a simple bimodal target distribution: a mixture of two univariate Gaussian distributions. The

aim is to sample from this distribution using parallel tempering with temperature adaptation.

We run the PT engine with $N = 4$ chains, with $T_{\text{low}} = 1$ so that the coldest chain samples from the target distribution and the remaining chains sample from progressively higher temperatures. We monitor the empirical Swap Acceptance Probability (SAP) between adjacent chains by recording the number of attempted and successful swaps throughout the simulation.

Figure 3.1 illustrates how the SAP evolves over time due to the adaptive temperature adjustment mechanism. The adaptation helps balance swap rates between all neighboring chains to match a target SAP, improving inter-chain communication.

Figure 3.2 displays a histogram of the samples obtained from the coldest chain. The histogram successfully recovers the bimodal structure of the target distribution, confirming that adaptive PT facilitates effective exploration of the state space and avoids mode trapping.

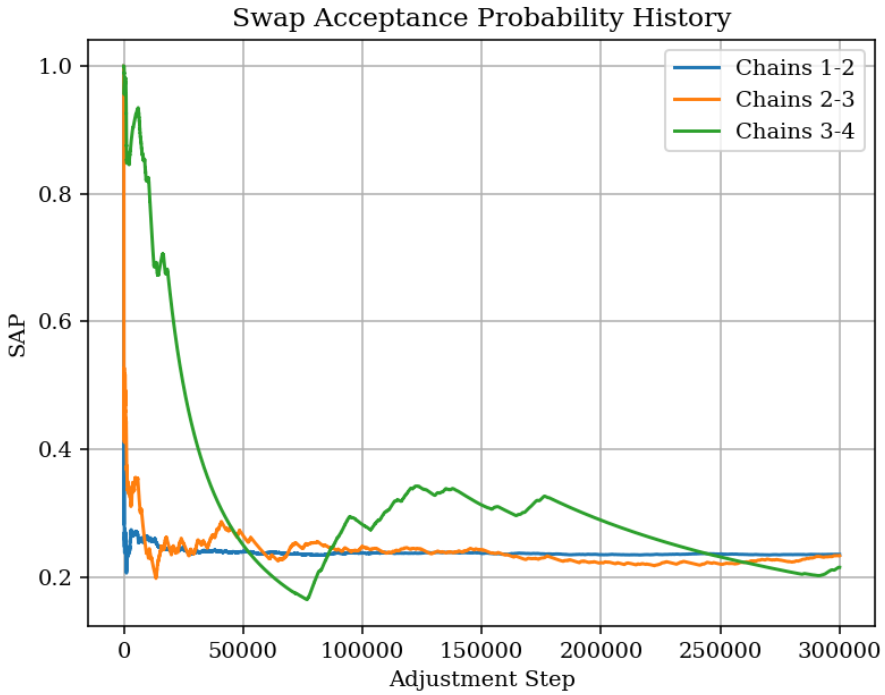

Figure 3.1: Empirical SAP evolution across iterations due to temperature adaptation between adjacent chains.

3.2.2 3-SAT

We now evaluate the adaptive temperature adjustment mechanism on a classical combinatorial optimization problem: 3-SAT. In this experiment, we consider a Boolean formula involving 10 variables and 100 clauses. Although the number of variables is relatively small, the large number of clauses makes the instance extremely constrained and difficult to satisfy.

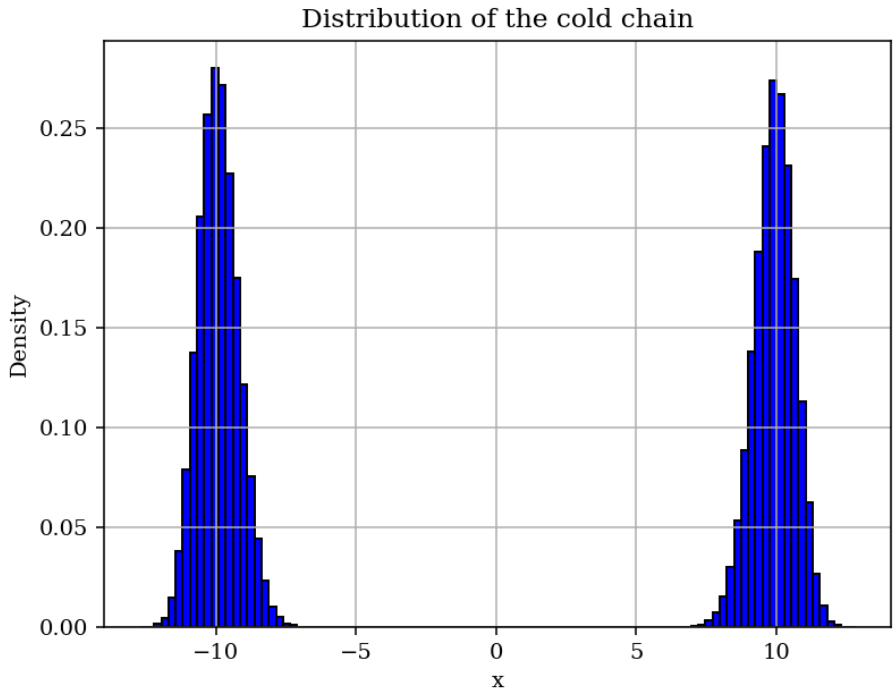

Figure 3.2: Histogram of samples from the cold chain, showing both modes of the target Gaussian mixture.

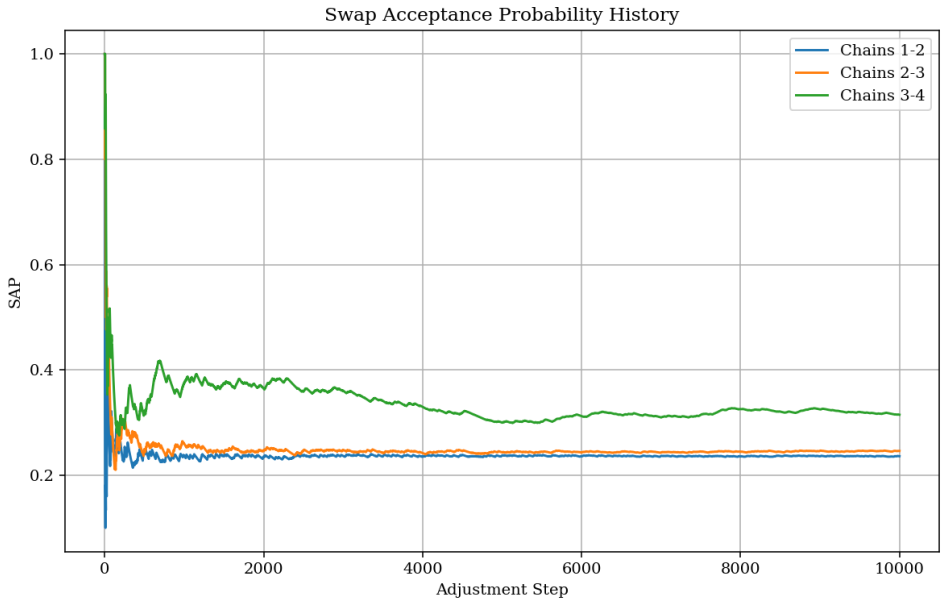

Figure 3.3: SAP evolution for the 3-SAT problem with 10 variables and 100 clauses using adaptive parallel tempering. The high clause-to-variable ratio makes the energy landscape rugged and highly multimodal, but adaptive temperature tuning improves inter-chain mixing.

In general, the 3-SAT problem is NP-complete, and its difficulty increases rapidly with the clause-to-variable ratio. In our case, the ratio is 10, which is far above the satisfiability threshold (approximately 4.27). At such high densities, most randomly generated formulas are unsatisfiable, and the energy landscape becomes rugged with many local minima and narrow basins of attraction.

We use $N = 4$ parallel tempering chains with $T_{\text{low}} = 0.1$, a low temperature chosen to focus the cold chain on low-energy (high-quality) assignments. The high-temperature chains facilitate exploration of the configuration space by helping the sampler escape local optima.

Figure 3.3 shows the SAP evolution over time. The adaptive temperature adjustment mechanism dynamically calibrates the inverse temperatures to balance swap probabilities between adjacent chains. This promotes efficient mixing across the chains, which is especially important in discrete problems like 3-SAT where the energy landscape is fragmented and difficult to traverse using standard MCMC methods.

Chapter 4

Neighbor Generation

4.1 State Space Analysis

We define a **state** following the conventional heuristic-based formulation for JSSP:
Definition 4.1.1. *A **state** is a poset represented by m chains, each corresponding to a machine's processing sequence of n operations:*

$$L_D = \{(s_{1_1}, s_{1_2}, \ldots, s_{1_n}), \ldots, (s_{m_1}, s_{m_2}, \ldots, s_{m_n})\},$$

where $(s_{i_1}, s_{i_2}, \ldots, s_{i_n})$ *is the operation order on machine* i*.*

For example, for instance 2.1.1:

$$L_D = \{(1, 8, 4), (2, 5, 7), (6, 3, 9)\},$$

indicating operations 1, 8, and 4 are scheduled on machine 1 in that order. This defines a schedule, though not necessarily a feasible one.

The subscript D indicates that L_D corresponds to directed disjunctive arcs in the activity graph. The conjunctive constraints remain fixed for given n and m, and can be represented as:

$$L_C = \{(1, 2, \ldots, m), \ldots, ((n-1)m + 1, \ldots, nm)\}.$$

Here, L_C and L_D represent topological orderings of two graphs over the same vertex set $\{1, \ldots, nm\}$: L_C corresponds to n linear chains, and L_D to m transitive tournaments. (We omit auxiliary nodes s and t, as they do not influence cycles and always appear at the beginning and end of any topological order)

Let $\mathcal{G}_C$ and $\mathcal{G}_D$ denote these graphs, with adjacency matrices C and D respectively. See Figures 4.1 and 4.2 for visualizations of $\mathcal{G}_C$ and $\mathcal{G}_D$, derived from the disjunctive graph in Figure 2.1.

We observe that L_D can be viewed as a maximal antichain covering of L_C, since $\mathcal{G}_C$ and $\mathcal{G}_D$ share no edges, which also, allows us to express the full graph's adjacency matrix as $A = C + D$. A valid state L_D implies that the union $L_C \cup L_D$ forms a

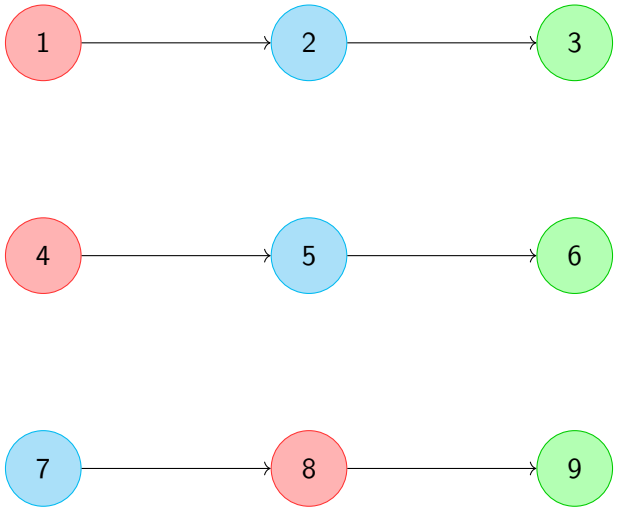

Figure 4.1: Conjunctive arcs graph corresponding to Figure 2.1.

poset. Any linear extension π of this union corresponds to a valid topological ordering of the activity graph. The permutation matrix P encoding π must simultaneously triangularize both C and D, i.e., $P^T AP = P^T CP + P^T DP$ is upper triangular only if both terms are individually upper triangular. Here, $C, D \in \{0,1\}^{nm \times nm}$.

Given a fixed operation-machine assignment, the state space size is $n!^m$, since each of the m machine chains in L_D can be ordered in $n!$ ways. However, the probability that a randomly sampled state is valid (i.e., leads to an acyclic union with L_C) is low. Our Monte Carlo simulations support this observation. For $n = m$, enumerating all possible machine assignments yields $(n!)^{n-1}$ options, and with $(n!)^n$ permutations per assignment, the total number of L_D configurations becomes intractable (e.g., $> 10^{10}$ even for $n = 5$).

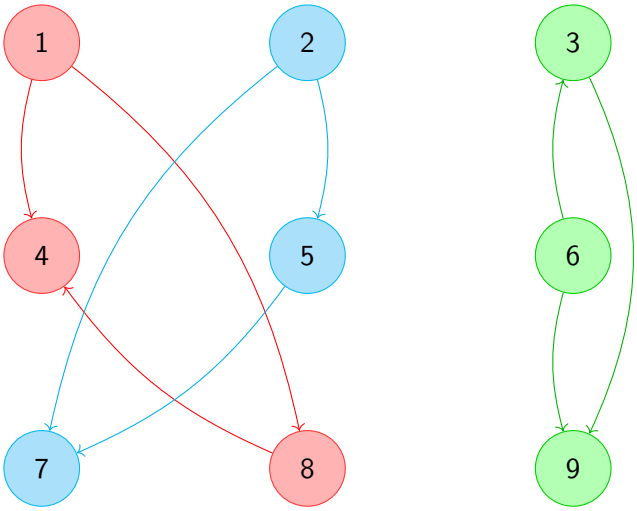

Figure 4.2: Oriented disjunctive arcs graph corresponding to Figure 2.1.

To estimate this probability, we sample 100,000 random permutations for a randomly chosen operation-machine mapping for each $n \in \{3, \ldots, 7\}$. We repeat

the experiment 100 times and compute the empirical probability that a state is valid. The results are shown in Figure 4.3.

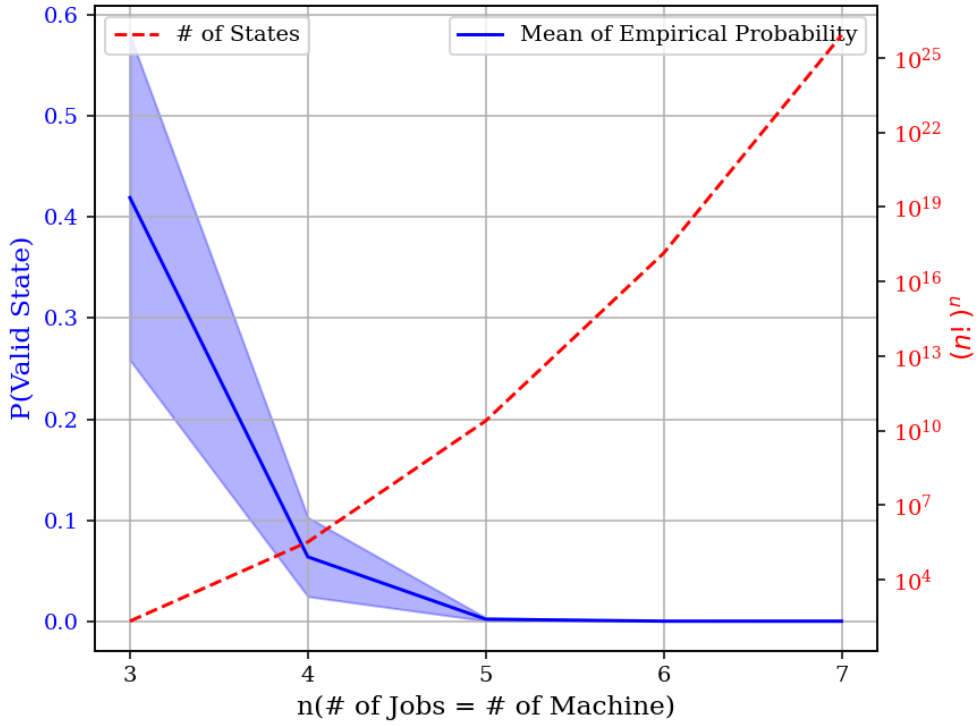

Figure 4.3: Empirical probability of a randomly sampled state being valid decreases with number of jobs.

4.2 Neighbor Generation

It is clear from the previous section that randomly permuting the operation assignments in a machine would lead to infeasible solutions most of the time, and the MCMC sampler has to be run for many iterations in order to effectively explore the space of feasible solutions. That is, for a given size of the space of feasible solutions, the amount of iterations required grows rapidly with the number of jobs and machines.

Thus, a clever way of handling neighbor generation that doesn't lead to an infeasible state most of the time is the need of the hour. Additionally, it should allow us to use an efficiently bounded incremental algorithm to recompute the longest path without doing it from scratch (which would be unbounded in terms of changes in input and output) at every single iteration.

Suppose we start with a feasible state L_D with topological ordering π and permutation matrix P encoding it, simultaneously triangularizing C and D. The graph $\mathcal{G}_D$ is a collection of transitive tournaments.

We pick a pair of adjacent operations corresponding to one of the machines in L_D. Now suppose we reverse their order:

$$L_D = \{\ldots, (\ldots, a, b, \ldots), \ldots\} \to L'_D = \{\ldots, (\ldots, b, a, \ldots), \ldots\}$$

This corresponds to a single edge flip in the corresponding transitive tournament. Since there are no nodes between these two neighbors in $\mathcal{G}_D$, we are guaranteed that this doesn't lead to a cycle in $\mathcal{G}_D$. We also interchange their corresponding positions in π. Now we are in a state L'_D with permutation matrix P' encoding π', only triangularizing D' and not C.

If we are able to move the nodes between a and b in π' such that we can now triangularize C without disturbing the triangularization of D, then L'_D is a valid state. This definitely happens when the submatrices

$$C'_{\text{sub}} = P'^T C P'[\pi'^{-1}(b) : \pi'^{-1}(a), \pi'^{-1}(b) : \pi'^{-1}(a)]$$
$$D'_{\text{sub}} = P'^T D P'[\pi'^{-1}(b) : \pi'^{-1}(a), \pi'^{-1}(b) : \pi'^{-1}(a)]$$

can be simultaneously triangularized.

It is trivial to show that when the dimensions of those submatrices are 2×2 or 3×3, they are always simultaneously triangularizable.

- When they are 2×2, C_{sub} would be a zero matrix, therefore they are trivially simultaneously triangularizable.
- When they are 3×3,

$$D'_{\text{sub}} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad C_{\text{sub}} \in \left\{ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \right\}$$

  and these satisfy

$$C_{\text{sub}}[C_{\text{sub}}, D'_{\text{sub}}] = 0 \quad \text{and} \quad [C_{\text{sub}}, D'_{\text{sub}}]D'_{\text{sub}} = 0$$

  and thus they are simultaneously triangularizable.

Intuitively, when the size of submatrices is 2, this implies that even in π they occur adjacent, and flipping them won't affect the topological sorting order. When the size is 3, in π there is one node between them. We know for sure it can't have the same machine as a or b. Therefore, we can move the in-between node either to the left or right to triangularize C_{sub}, and hence C, without disturbing D.

This gives a natural way to generate a valid neighbor, and we get the topological sorting order of the new neighbor with little to no effort. We can use this information to recompute the longest path efficiently.

So, we look for pairs of nodes that occur adjacent in L_D and are separated by at most one node in π, and swap them. But when swapping a, b with no nodes between them, it leads to a state where again b, a have no nodes between them. Since our plan is to use a priority queue to keep track of such nodes, we may end up swapping b, a again and end up in a loop.

We could handle this with clever bookkeeping, but in this work, we choose to swap adjacent nodes which are separated by only one node in π, so after the swap in the final topological ordering, they have no nodes between them and we don't end up choosing them again.

If no such pair exists, or occasionally with probability ϵ, we revert to randomly swapping two adjacent nodes on a randomly selected machine in L_D and restart the process.

A natural question arises: how frequently do these pairs occur? Because if they are too rare, then it is even worse than randomly swapping adjacent pairs, as we would be doing it randomly every time, but with extra bookkeeping to run this method.

Proposition 1. *The probability that there exists at least one pair of adjacent operations in a valid disjunctive set L_D that appear with exactly one operation between them in any linear extension of the union $L_C \cup L_D$ increases with the number of jobs and machines.*

Deriving an explicit analytical bound for this probability is deemed too difficult. The dependencies imposed by both conjunctive and disjunctive constraints result in intricate precedence structures.

To support the proposition, we employ Monte Carlo simulation. For varying values of n (the number of jobs, equal to the number of machines m), we randomly generate feasible disjunctive orderings L_D, compute corresponding linear extensions of the union $L_C \cup L_D$, and empirically estimate the frequency of the specified adjacency pattern. As noted earlier, obtaining valid states becomes increasingly difficult with larger n, so we restrict our experiments to $n \leq 6$. Each experiment is repeated 100 times to obtain a reliable estimate. The result of the monte carlo experiment in shown in Figure 4.4.

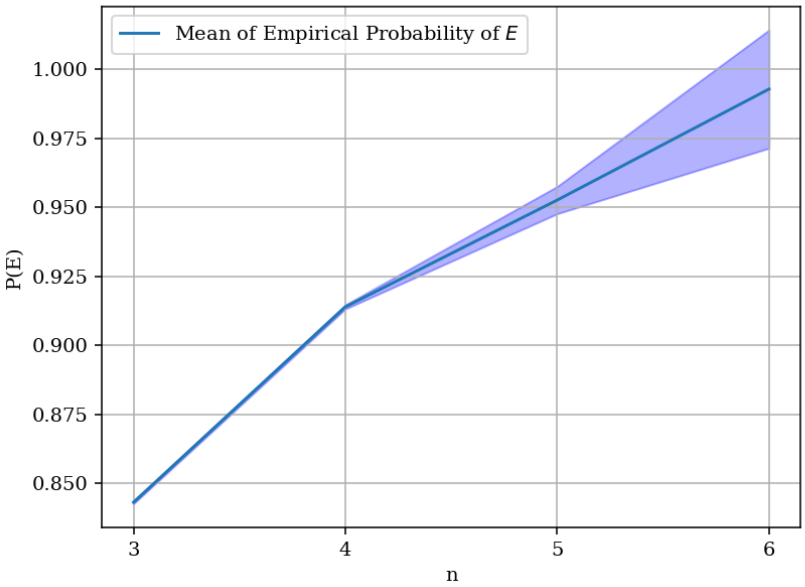

Figure 4.4: Monte Carlo estimate of the probability of encountering adjacent operations in L_D that appear with exactly one operation between them in a linear extension of $L_C \cup L_D$. The observed trend supports the claim that this probability increases with problem size (i.e., number of jobs and machines).

Algorithm 3 outlines the initialization steps required to set up the necessary data structures for neighbor generation, while Algorithm 4 details the complete procedure for generating neighbors and performing incremental topological updates.

Algorithm 3 Initialization for Neighbor Generation

Initialize priority queue Q
Initialize hash maps of hash maps `out` and `in`
for each machine in L_D **do**
    for each adjacent operation pair (a, b) **do**
        Compute $dist_\pi(a, b)$ ▷ Nodes between a and b in π
        `out`$[a]\,[b] \leftarrow dist_\pi(a, b)$
        `in`$[b]\,[a] \leftarrow dist_\pi(a, b)$
        if $dist_\pi(a, b) \neq 2$ **then**
            Push $(dist_\pi(a, b), (a, b))$ into Q
        else
            Push $(0, (a, b))$ into Q
        end if
    end for
end for

Algorithm 4 Neighbor Generation Block

Input: Initial machine schedule L_D, topological order π, ϵ
Output: A new neighboring schedule (L'_D, π')

Sample x from $Uniform[0, 1]$
Pop $(d, (a, b))$ from Q
if $x > \epsilon$ and $d = 0$ **then**
    Flip a and b in L_D
    Swap a and b in π
    Let c be the operation between a and b in the original π
    if c is in the same job as a, or c is in a different job from both a and b **then**
        Reorder π: $a\,c\,b \rightarrow b\,a\,c$
    else
        Reorder π: $a\,c\,b \rightarrow c\,b\,a$
    end if
    Compute Δ_i for each $i \in \{a, b, c\}$ ▷ Position change in π
    for each $i \in \{a, b, c\}$ **do**
        for each incoming node u of i **do**
            Update $\texttt{in}[i][u]$ and $\texttt{out}[u][i]$ using Δ_i
            if $\texttt{in}[i][u] \neq 2$ **then**
                Push $(\texttt{in}[i][u], (u, i))$ into Q
            else
                Push $(0, (u, i))$ into Q
            end if
        end for
        for each outgoing node v of i **do**
            Update $\texttt{in}[v][i]$ and $\texttt{out}[i][v]$ using Δ_i
            if $\texttt{in}[v][i] \neq 2$ **then**
                Push $(\texttt{in}[v][i], (i, v))$ into Q
            else
                Push $(0, (i, v))$ into Q
            end if
        end for
    end for
    Recompute longest path for v for each node v appearing after b in updated π
    return Updated neighbor (L'_D, π')
else
    Randomly swap two adjacent nodes on a randomly selected machine in L_D and restart the process
end if

Chapter 5

Results and Conclusion

5.1 Implementation Details

To achieve efficient performance and scalability, the complete algorithm was implemented in C++. Given the computationally intensive nature of our problem — involving numerous MCMC iterations and replica interactions — choosing C++ enabled us to optimize runtime significantly compared to interpreted alternatives. To leverage multi-core processing capabilities and accelerate convergence, we employed OpenMP. Each replica in the parallel tempering framework was assigned to a separate thread, allowing concurrent execution of independent Markov chains.

The number of replicas was set to 8, providing a balance between exploration capability and computational overhead. A smaller number of replicas would reduce the diversity of explored solutions, while a larger number would increase synchronization and computational costs. The minimum temperature was fixed at 0.01. The decay function parameters were set as $C = 0.9$ and $\eta = 0.7$.

We set the number of MCMC sweeps per replica $N_{\text{sweep}} = 100$. This ensures each replica has ample time to explore its subspace before interacting with other chains. The probability ϵ, which introduces randomness to encourage exploration, was set to 0.23.

For benchmarking and empirical validation, we used instances from the Taillard Job Shop Scheduling Problem (JSSP) dataset, a widely recognized benchmark suite in scheduling research. These instances vary in size and difficulty and are commonly used for comparing optimization methods. We evaluated our method against the Google OR-Tools, a state-of-the-art constraint solver for job shop scheduling. The comparative results are discussed in the next section.

5.2 Comparison with State of the Art

Table 5.1 lists the Taillard instances used to benchmark our proposed method against Google OR-Tools. The problem instances increase in size as we go down the table, starting from the relatively smaller (yet still challenging) 30×15 instances. Even

these moderate-sized instances pose significant challenges for many solvers due to the combinatorial explosion of possible schedules.

Both our implementation and Google OR-Tools were allowed a runtime budget of 5 minutes (300 seconds) for each instance. We report the makespan computed by each solver and its discrepancy from the best-known upper bound for that instance. This comparison highlights the effectiveness and competitiveness of our approach.

Taillard Instance	AdapPT		Google-OR Tools	
	Solution	Optimality Gap %	Solution	Optimality Gap %
ta32 (30×15)	2011	9.2	1900	3.2
ta34 (30×15)	1960	5.8	1898	2.5
ta41 (30×20)	2277	10.3	2301	11.4
ta42 (30×20)	2101	5.9	2042	2.9
ta51 (50×15)	3458	25.3	3279	18.8
ta52 (50×15)	3325	20.6	3171	15.0
ta61 (50×20)	3156	10.0	3198	11.8
ta32 (50×20)	2921	6.0	3117	13.1
ta62 (100×20)	5496	6.0	5467	5.5
ta63 (100×20)	6057	8.7	5960	7.0

Table 5.1: Comparison of makespan and percentage discrepancy from best-known upper bound for selected Taillard instances using our method and Google OR-Tools (300s runtime).

5.3 Future Work

- The parameters of the decay function $\gamma(i)$—namely C and η—were held fixed across all chains. However, preliminary experiments indicate that using smaller values of C and larger values of η for low-temperature chains (and the reverse for high-temperature chains) can lead to better performance. Adapting these parameters dynamically could improve convergence.
- The current method primarily focuses on adjacent node swaps. This approach can be generalized to handle swaps involving nodes separated by multiple intermediate operations. A promising direction is to construct subgraphs $\mathcal{G}_{C_{\text{sub}}+D_{\text{sub}}}$ induced by a small number of nodes k between the target nodes. By computing a topological sort of these smaller subgraphs ($k < n$), we can perform efficient local updates while preserving feasibility.
- Provide a lower bound estimate for Proposition 1. While deriving an exact expression may be challenging, a reasonable rough lower bound can still be established.
- The existing implementation can be further optimized by improving data movement, storage patterns, and better leveraging OpenMP for parallelism. This would significantly reduce overhead and improve scalability.

5.4 Conclusion

In this work, we developed an adaptive parallel tempering framework tailored for the Job Shop Scheduling Problem (JSSP). Our approach introduces a novel temperature update rule designed to maintain optimal swap acceptance probabilities, thereby enhancing exploration across the complex energy landscape. Additionally, we proposed a new method for generating valid neighbors in the solution space, coupled with an efficient procedure to recalculate longest paths after each move, ensuring solution feasibility and accurate evaluation.

We benchmarked our framework against the state-of-the-art solver Google OR-Tools on challenging JSSP instances, demonstrating competitive performance in terms of solution quality for a given time budget. These results highlight the potential of adaptive parallel tempering combined with domain-specific neighbor generation strategies to effectively tackle combinatorial scheduling problems.

References

[1] H. Xiong, S. Shi, D. Ren, and J. Hu, "A survey of job shop scheduling problem: The types and models," *Computers Operations Research*, vol. 142, p. 105731, 2022.

[2] F. Garza-Santisteban, R. Sánchez-Pámanes, L. A. Puente-Rodríguez, I. Amaya, J. C. Ortiz-Bayliss, S. Conant-Pablos, and H. Terashima-Marín, "A simulated annealing hyper-heuristic for job shop scheduling problems," in *2019 IEEE Congress on Evolutionary Computation (CEC)*, pp. 57–64, 2019.

[3] D. Vivekanandan, S. Wirth, P. Karlbauer, and N. Klarmann, "A reinforcement learning approach for scheduling problems with improved generalization through order swapping," *Machine Learning and Knowledge Extraction*, vol. 5, no. 2, pp. 418–430, 2023.

[4] I. G. Smit, J. Zhou, R. Reijnen, Y. Wu, J. Chen, C. Zhang, Z. Bukhsh, Y. Zhang, and W. Nuijten, "Graph neural networks for job shop scheduling problems: A survey," 2024.

[5] J. Zhang, G. Lo Bianco, and J. C. Beck, "Solving job-shop scheduling problems with qubo-based specialized hardware," *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 32, pp. 404–412, Jun. 2022.

[6] F. Garza-Santisteban, R. Sánchez-Pámanes, L. A. Puente-Rodríguez, I. Amaya, J. C. Ortiz-Bayliss, S. Conant-Pablos, and H. Terashima-Marín, "A simulated annealing hyper-heuristic for job shop scheduling problems," in *2019 IEEE Congress on Evolutionary Computation (CEC)*, pp. 57–64, 2019.

[7] L. Hernández-Ramírez, J. Frausto Solis, G. Castilla-Valdez, J. J. González-Barbosa, D. Terán-Villanueva, and M. L. Morales-Rodríguez, "A hybrid simulated annealing for job shop scheduling problem," *International Journal of Combinatorial Optimization Problems and Informatics*, vol. 10, p. 6–15, Aug. 2018.

[8] D. C. Bissoli, W. A. S. Altoe, G. R. Mauri, and A. R. S. Amaral, "A simulated annealing metaheuristic for the bi-objective flexible job shop scheduling problem," in *2018 International Conference on Research in Intelligent and Computing in Engineering (RICE)*, pp. 1–6, 2018.

[9] P. V. Matrenin, "Improvement of ant colony algorithm performance for the job-shop scheduling problem using evolutionary adaptation and software realization heuristics," *Algorithms*, vol. 16, no. 1, 2023.

[10] A. Kone and D. A. Kofke, "Selection of temperature intervals for parallel-tempering simulations," *The Journal of Chemical Physics*, vol. 122, p. 206101, 05 2005.

[11] B. Miasojedow, E. Moulines, and M. Vihola, "Adaptive parallel tempering algorithm," 2012.

[12] W. D. Vousden, W. M. Farr, and I. Mandel, "Dynamic temperature selection for parallel tempering in markov chain monte carlo simulations," *Monthly Notices of the Royal Astronomical Society*, vol. 455, pp. 1919–1937, 11 2015.

[13] K. Dabiri, M. Malekmohammadi, A. Sheikholeslami, and H. Tamura, "Replica exchange mcmc hardware with automatic temperature selection and parallel trial," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 7, pp. 1681–1692, 2020.

[14] M. A. Bender, J. T. Fineman, and S. Gilbert, "A new approach to incremental topological ordering," in *Proceedings of the Twentieth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '09, (USA), p. 1108–1115, Society for Industrial and Applied Mathematics, 2009.

[15] S. McCauley, B. Moseley, A. Niaparast, and S. Singh, "Incremental topological ordering and cycle detection with predictions," 2024.

[16] I. Katriel, L. Michel, and P. V. Hentenryck, "Maintaining longest paths incrementally," *Constraints*, vol. 10, no. 2, pp. 159–183, 2005.

[17] G. Madraki and R. P. Judd, "Recalculating the length of the longest path in perturbed directed acyclic graph," *IFAC-PapersOnLine*, vol. 52, no. 13, pp. 1560–1565, 2019. 9th IFAC Conference on Manufacturing Modelling, Management and Control MIM 2019.

[18] G. Madraki and R. Judd, "Accelerating the calculation of makespan used in scheduling improvement heuristics," *Computers Operations Research*, vol. 130, p. 105233, 2021.

[19] G. Madraki, S. Mousavian, and Y. S. and, "A theoretical framework to accelerate scheduling improvement heuristics using a new longest path algorithm in perturbed dags," *International Journal of Production Research*, vol. 61, no. 3, pp. 818–838, 2023.

[20] G. Ramalingam, *Bounded incremental computation*. PhD thesis, USA, 1993. UMI Order No. GAX93-31279.

[21] D. Shemesh, "A simultaneous triangularization result," *Linear Algebra and its Applications*, vol. 498, pp. 394–398, 2016. Special Issue: Legacy of Hans Schneider.

[22] G. O. Roberts and J. S. Rosenthal, "Coupling and ergodicity of adaptive markov chain monte carlo algorithms," *Journal of Applied Probability*, vol. 44, no. 2, p. 458–475, 2007.